

WebApp - Arquitetura MVC

Resumo conceitual da organização do projeto Flask

1. Objetivo

Este documento registra a comparação entre o padrão MVC (Model-View-Controller) e a estrutura utilizada no projeto WebApp. O objetivo é documentar como as responsabilidades da aplicação estão distribuídas e facilitar a manutenção e evolução do projeto.

2. Correspondência entre MVC e WebApp

MVC	WebApp	Responsabilidade
Model	models/ database/	Representação dos dados, entidades, persistência e comunicação com o banco de dados.
View	templates/ static/	Interface apresentada ao usuário: HTML, CSS, JavaScript, imagens e demais recursos visuais.
Controller	routes/	Recebe as requisições HTTP, coordena o fluxo da aplicação e determina a resposta ou página a ser exibida.
Bootstrap / configuração	app.py	Inicializa e configura a aplicação Flask, registra Blueprints e conecta os componentes principais.

A associação simplificada pode ser representada da seguinte forma:

```
Model -> database/ + models/  
View -> templates/ + static/  
Controller -> routes/  
Bootstrap -> app.py
```

3. Papel do app.py

Embora seja possível associar inicialmente **routes/ + app.py** ao Controller, é mais preciso separar as responsabilidades. As rotas exercem efetivamente o papel de Controller, enquanto o **app.py** funciona como ponto de inicialização e composição da aplicação.

No WebApp, o app.py é responsável principalmente por criar a aplicação Flask, carregar configurações necessárias e registrar os Blueprints que organizam as rotas.

4. Fluxo típico da aplicação

Um fluxo simplificado de uma operação, como autenticação ou consulta de dados, pode ser visualizado assim:

```
Navegador  
|  
v  
routes/ Controller  
|  
v  
models/ Model  
|  
v  
database/ Persistência  
|  
v
```

```
routes/
|
v
templates/ View
|
v
Navegador
```

5. Estrutura conceitual atual

A estrutura principal do projeto pode ser entendida da seguinte maneira:

```
webapp/
|-- app.py # Inicialização e composição
|-- database/ # Infraestrutura e acesso ao banco
|-- models/ # Model
|-- routes/ # Controller
|-- templates/ # View - HTML
`-- static/ # View - CSS, JS, imagens etc.
```

6. MVC como separação de responsabilidades

MVC não exige obrigatoriamente três diretórios chamados Model, View e Controller. O princípio mais importante é a separação das responsabilidades. O WebApp segue esse conceito ao evitar que a interface, o acesso aos dados e o tratamento das requisições fiquem concentrados em um único módulo.

7. Possível evolução da arquitetura

Com o crescimento do WebApp, novas camadas podem ser introduzidas, por exemplo **services/** para regras de negócio e **repositories/** para concentrar operações de persistência. Nesse cenário, o projeto continua utilizando os princípios do MVC, mas passa a adotar uma arquitetura em camadas mais explícita.

```
View (templates/static)
|
Controller (routes)
|
Services (regras de negócio)
|
Repositories (persistência)
|
Models / Database
```

8. Conclusão

A estrutura atual do WebApp pode ser classificada de forma coerente dentro do padrão MVC:

models/database representam a camada de Model, **templates/static** representam a View e **routes** representam o Controller. O **app.py** deve ser entendido principalmente como o ponto de inicialização e composição da aplicação. Essa organização fornece uma base simples, clara e adequada para a evolução futura do projeto.